

Sistema Autónomo de Protección y Monitoreo de Café con Diagnóstico Inteligente Multimodal

Franco Arias, Mauricio
Universidad Nacional Federico Villarreal
Lima, Perú
Palacios Hinojosa, Lesli
Universidad Nacional Federico Villarreal
Lima, Perú

Orellana Fernández, Alexandra
Universidad Nacional Federico Villarreal
Lima, Perú
Olivera Torres, Leonardo
Universidad Nacional Federico Villarreal
Lima, Perú

Resumen— El presente artículo describe el diseño y desarrollo de un sistema inteligente autónomo para la protección y control del secado de café, orientado a mitigar pérdidas económicas por variaciones climáticas inesperadas. El sistema se fundamenta en una arquitectura de control basada en el microcontrolador ATmega328P, que procesa en tiempo real datos de sensores de lluvia, temperatura y humedad ambiental para automatizar una cobertura protectora. Como innovación técnica, se integra una interfaz móvil de monitoreo orientada a IoT que incorpora herramientas de análisis multimodal mediante inteligencia artificial (IA), permitiendo la interpretación visual del estado del grano y la generación de recomendaciones técnicas para el productor. Los resultados experimentales confirman un tiempo de respuesta ante eventos críticos menor a 2.5 segundos; adicionalmente, el módulo visual alcanzó una precisión global de 90 % en un conjunto de 40 imágenes. La solución propuesta protege el café durante el secado y aporta un diagnóstico preventivo basado en datos para mejorar la estandarización del proceso.

Palabras clave: Arduino, Agricultura inteligente, Internet de las Cosas (IoT), Inteligencia Artificial (IA), Secado de café.

I. INTRODUCCIÓN

En el sector cafetalero, el proceso de secado es una etapa crítica que determina la calidad final del producto. Tradicionalmente, este proceso se realiza de manera artesanal mediante exposición directa al sol, lo que hace que el grano sea altamente vulnerable a condiciones climáticas variables como lluvias inesperadas, alta humedad ambiental y elevadas temperaturas, las cuales afectan las propiedades sensoriales del café y favorecen la proliferación de hongos como el *Aspergillus* [1], [2]. Estudios técnicos indican que temperaturas iguales o superiores a 40°C durante este proceso pueden degradar significativamente características esenciales como el aroma y el sabor, disminuyendo su valor comercial [3].

A pesar de esta problemática, muchos productores carecen de sistemas automatizados que permitan proteger el café ante cambios climáticos repentinos [3]. Ante esta brecha, el presente proyecto diseña y desarrolla un sistema inteligente de protección y monitoreo basado en condiciones climáticas. El sistema integra sensores de lluvia, temperatura y humedad ambiental, utilizando el microcontrolador ATmega328P para ejecutar una lógica de control autónoma que automatiza el despliegue de la cobertura protectora.

Como evolución significativa, se ha incorporado una arquitectura de monitoreo móvil que permite la supervisión remota y aproxima el sistema a un enfoque de agricultura de precisión [4]. Asimismo, la integración de inteligencia artificial (IA) para el diagnóstico visual del grano favorece una toma de decisiones informada, transformando el prototipo de un mecanismo de protección pasiva a una plataforma integral de gestión de calidad [2], [4].

II. DISEÑO

El sistema se basa en una arquitectura de hardware modular centrada en el microcontrolador ATmega328P. El diseño garantiza una respuesta autónoma ante variaciones climáticas, minimizando el riesgo de degradación del café.

A. Arquitectura de Hardware y Conexiones

La arquitectura del sistema integra sensores, actuadores y una unidad central de procesamiento. Las conexiones físicas fueron validadas mediante simulación y prototipado, garantizando la integridad de las señales.

TABLA I
Especificaciones de pines y funciones del sistema

Componente	Pin asignado	Tipo de señal	Función dentro del sistema
Sensor de lluvia	D2	Entrada digital	Detecta presencia de lluvia. En el código, lluvia se identifica cuando la lectura es LOW.
Sensor DHT22	A0	Entrada digital	Lee la temperatura ambiental mediante la librería DHT. Aunque usa A0, funciona como pin digital.
Puente H - IN1	D3	Salida digital	Controla una dirección de giro del motor del techo.
Puente H - IN2	D4	Salida digital	Controla la dirección opuesta de giro del motor del techo.
Puente H - ENA	D5	Salida PWM	Activa el motor y regula su velocidad mediante analogWrite.
LCD I2C - SDA	A4	Comunicación I2C	Línea de datos para la pantalla LCD.
LCD I2C - SCL	A5	Comunicación I2C	Línea de reloj para la pantalla LCD.
Alimentación	5 V / GND	Energía	Alimenta los sensores, la pantalla LCD y los módulos conectados.

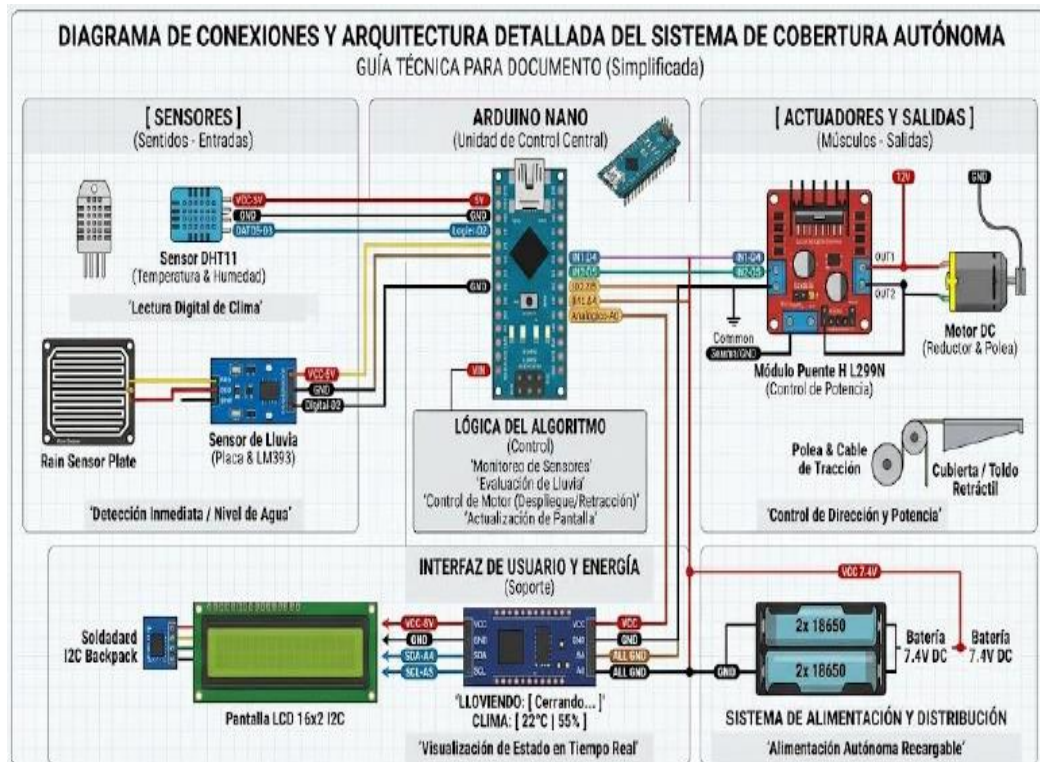


Fig. 1 Diagrama de conexiones y arquitectura detallada del sistema de cobertura autónoma.

B. Lógica de Control y Máquina de Estados

La lógica de control se basa en una Máquina de Estados Finita (FSM), la cual permite gestionar los estados: Techo Abierto, Techo Cerrado por Temperatura y Techo Cerrado por Lluvia. La FSM garantiza que el sistema priorice la protección contra la lluvia, independientemente de la temperatura ambiental.

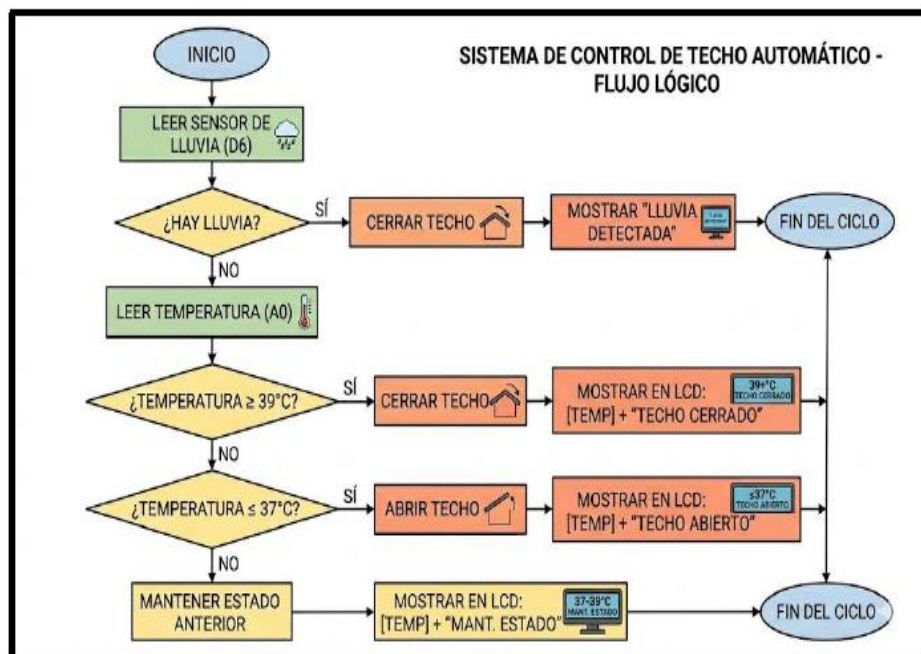


Fig. 2 Diagrama de flujo lógico del sistema de control.

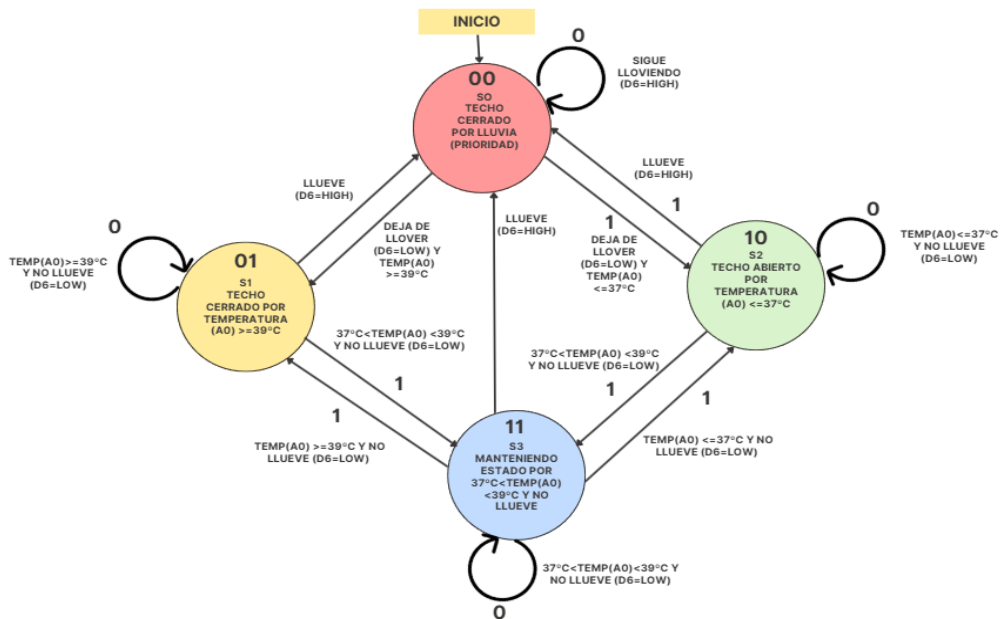


Fig. 3 Representación del grafo de estados de la máquina de estados finita.

TABLA II

ESTADOS	Q1	Q0	C	A
S0	0	0	1	0
S1	0	1	1	0
S2	1	0	0	1
S3	1	1	0	0

Matriz de salidas y transiciones de estados

C. Gestión de Memoria y Procesamiento

El ATmega328P ejecuta el programa mediante un ciclo de instrucción estándar (Fetch-Decode-Execute), optimizando el uso de la memoria SRAM (2 KB) para el almacenamiento temporal de variables como la temperatura y el estado de la cobertura.

TABLA III
Recursos de memoria utilizados por el sistema.

Recurso medido	Valor aproximado	Porcentaje de uso	Observación
Memoria Flash	9842 bytes	30.5 %	Almacena el programa y las librerías utilizadas.
Memoria SRAM	683 bytes	33.3 %	Se usa para variables, objetos y datos temporales.
Memoria EEPROM	0 bytes	0 %	El código no la utiliza.

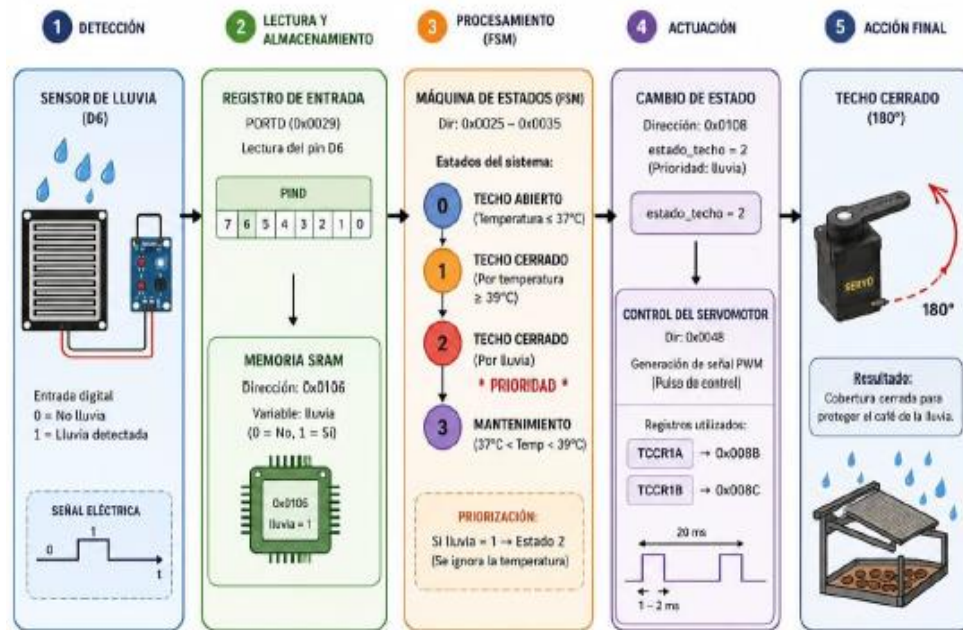


Fig. 4 Diagrama del ciclo de ejecución (Fetch-Decode-Execute) implementado en el sistema.

D. Arquitectura de la Aplicación Móvil e Integración con IA

La aplicación móvil constituye la interfaz de monitoreo e interpretación del sistema. Su función principal es recibir los datos generados por el prototipo físico, presentarlos al usuario de forma comprensible y complementar la información ambiental con un análisis visual asistido por inteligencia artificial.

La arquitectura de software se organiza en cinco etapas. En primer lugar, el Arduino obtiene las lecturas de los sensores de temperatura, humedad ambiental y lluvia. Luego, estos datos son enviados mediante el módulo Bluetooth HC-05 hacia la aplicación móvil. Una vez recibida la información, la app muestra en tiempo real las variables ambientales y el estado de la cobertura. De manera complementaria, el usuario puede capturar una imagen del café

ubicado en la bandeja de secado. Esta imagen, junto con los datos ambientales registrados, es enviada al servicio de inteligencia artificial Gemini para generar una interpretación textual del estado observado.

El resultado del análisis no se limita a mostrar valores numéricos, sino que transforma los datos en información útil para el usuario. Por ejemplo, la aplicación puede describir la distribución visible del café, diferencias de coloración, presencia de objetos extraños sobre la superficie y relacionar estas observaciones con la temperatura, humedad ambiental, lluvia y estado de la cobertura. Posteriormente, cada evento puede almacenarse en una bitácora digital, donde se registra la fecha, hora, imagen capturada, mediciones de sensores y análisis generados. A partir de estos registros, la aplicación permite visualizar gráficos de tendencia de temperatura y humedad, facilitando el seguimiento del proceso de secado.

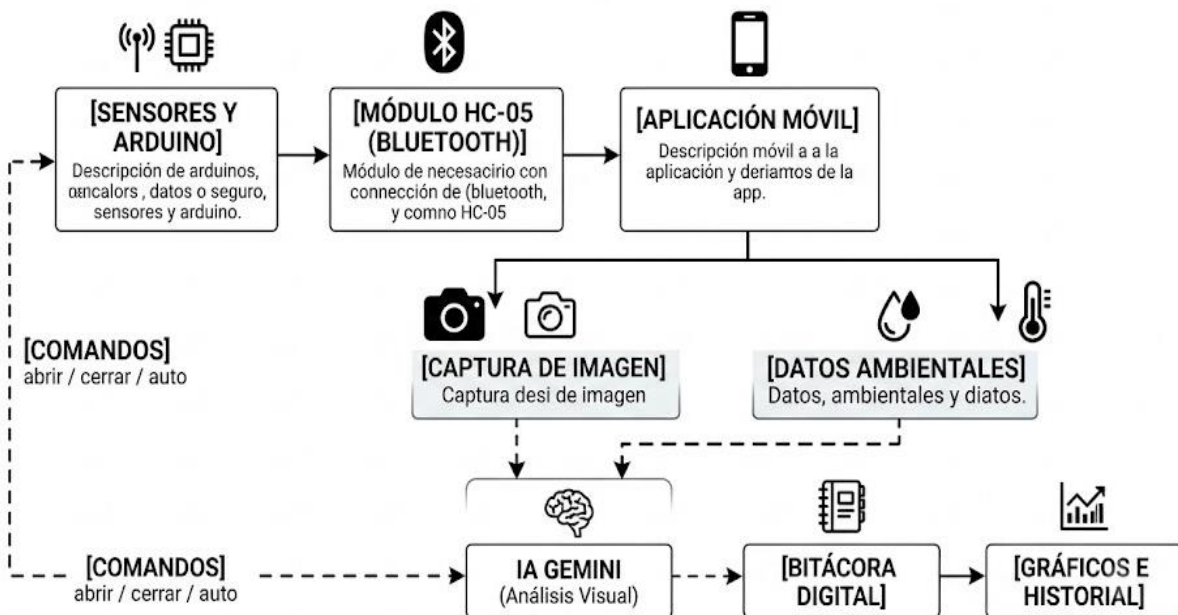


Fig. 5 Flujo de la aplicación móvil e integración con inteligencia artificial.

Esta arquitectura permite que el sistema no solo actúe de manera automática sobre la cobertura, sino que también registre, interprete y comunique la información del proceso de secado de forma más accesible para el productor.

III. RESULTADOS EXPERIMENTALES
A. Validación del Sistema de Control (Métricas de Hardware)

Las pruebas comparativas entre la simulación y el entorno físico confirmaron la fiabilidad de la lógica de control. El tiempo de respuesta del sistema desde la detección de una condición crítica (lluvia o exceso de temperatura) hasta la acción mecánica (cierre del techo) fue medido satisfactoriamente.

TABLA IV
Comparativa de desempeño entre simulación y hardware real

Métrica	Simulación	Hardware físico	Justificación
Punto de Cierre (≥ 39 °C)	Reacción exactamente a los 39.0 °C	Activación a 39 °C	El sistema cerró la cobertura al alcanzar el umbral programado.

Métrica	Simulación	Hardware físico	Justificación
Punto de Reapertura (≤ 37 °C)	Reacción exactamente a los 37.0 °C	Activación a 37 °C	El sistema reabrió la cobertura al regresar al umbral seguro.
Tiempo de Respuesta Mecánica	0 segundos (Cambio de estado instantáneo)	1.5 segundos	Tiempo necesario para completar el movimiento mecánico de apertura o cierre.

B. Validación del Análisis Inteligente (Métricas de la App con IA)

La integración de la IA se evaluó mediante la capacidad del sistema para correlacionar los datos de temperatura/humedad con la imagen del cultivo. La aplicación demostró ser eficaz al generar recomendaciones técnicas basadas en el análisis de variables ambientales y visuales.



Fig. 6 Interfaz de la aplicación con datos en vivo de temperatura y humedad y estado de conexión.



Fig. 7 Informe generado por IA: diagnóstico y recomendación técnica tras el análisis visual.

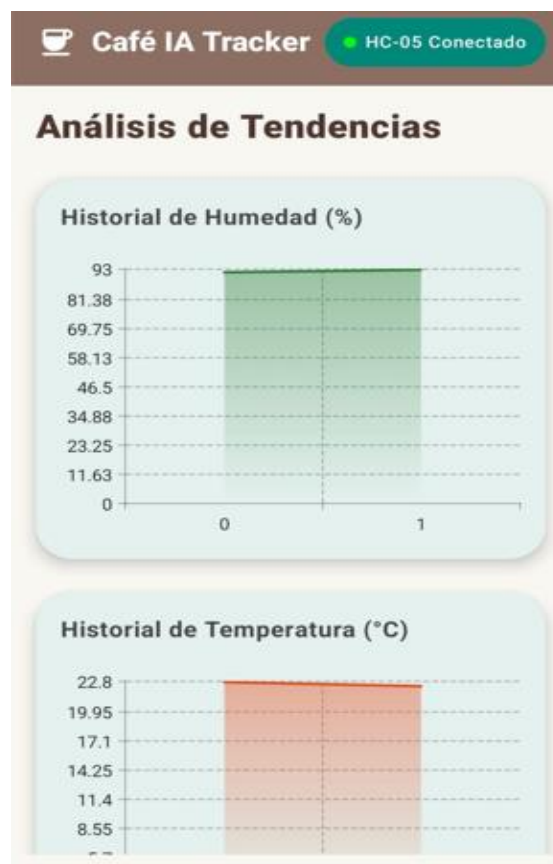


Fig. 8 Gráficos del historial de temperatura y humedad.

TABLA V
Métricas de desempeño del sistema

Métrica	Valor obtenido	Descripción
Tiempo promedio de respuesta del sistema	1.5-2.5 s	Tiempo transcurrido desde la detección de lluvia o temperatura crítica hasta el movimiento completo del techo.
Punto de activación por temperatura	$\geq 39\text{ }^{\circ}\text{C}$	Temperatura a la que el sistema activa automáticamente el cierre de la cobertura durante las pruebas realizadas.
Punto de reapertura por temperatura	$\leq 37\text{ }^{\circ}\text{C}$	Temperatura a la que el sistema abre nuevamente la cobertura al regresar a una condición segura.
Detección de lluvia	100 %	El sensor detectó correctamente la presencia de lluvia en todas las pruebas realizadas y activó el cierre inmediato de la cobertura.
Tiempo de respuesta ante lluvia	Menor a 2 s	Tiempo requerido para detectar la lluvia e iniciar el cierre del techo.
Monitoreo de humedad ambiental	En tiempo real	La aplicación mostró continuamente los valores reportados por el sensor DHT22 durante el proceso de secado.
Tiempo de actualización de la aplicación	1 s	Intervalo aproximado de actualización de los datos de temperatura, humedad y estado de la cobertura.
Tiempo de respuesta del análisis con IA	3.5 s	Tiempo aproximado requerido para recibir el diagnóstico generado por Gemini después de enviar la imagen.
Uso de memoria flash	30.5 %	Espacio ocupado por el programa y las librerías del sistema.
Uso de memoria SRAM	33.3 %	Memoria utilizada para variables y almacenamiento temporal durante la ejecución.

Métrica	Valor obtenido	Descripción
Disponibilidad del sistema durante las pruebas	100 %	El sistema respondió correctamente en todas las pruebas experimentales realizadas, sin fallos de funcionamiento.

Posteriormente, se realizó una evaluación global del desempeño del prototipo considerando tanto el comportamiento del sistema de control como el funcionamiento de la aplicación móvil. Los resultados obtenidos muestran que el tiempo de respuesta mecánica se mantuvo dentro del rango esperado para proteger el proceso de secado, mientras que la aplicación permitió visualizar las variables ambientales prácticamente en tiempo real y generar diagnósticos mediante inteligencia artificial en pocos segundos. Asimismo, el bajo porcentaje de utilización de la memoria del microcontrolador evidencia que la solución dispone de recursos suficientes para incorporar futuras funcionalidades sin comprometer el rendimiento del sistema.

C. Análisis de Resultados

El sistema de control presenta una respuesta determinista y coherente. Mientras que en simulación la respuesta es instantánea, en el hardware real se registró una latencia promedio de 1.5 a 2.5 segundos, valor compatible con el objetivo de respuesta definido para el prototipo. La integración de la IA añade una capa de análisis que permite al productor revisar tendencias históricas y tomar medidas correctivas antes de que se presenten daños en el café.

D. Protocolo de Validación del Modelo de Visión por Computadora

Para evaluar la capacidad analítica del módulo de inteligencia artificial, se definió un protocolo de validación basado en imágenes y datos ambientales. La arquitectura combina un preprocesamiento local mediante el módulo CoffeeColorAnalyzer.kt, que extrae características cromáticas de la imagen capturada, con el análisis semántico realizado en la nube. Este procedimiento permite sustentar las recomendaciones en patrones visuales observables, sin asumir que la respuesta del modelo sea determinista.

El protocolo consideró muestras con diferentes niveles de humedad, sobreexposición al calor y presencia de impurezas. Las predicciones del sistema se compararon con una inspección técnica de control para cuantificar el número de aciertos por condición evaluada.

E. Evaluación de la Precisión del Modelo de Visión por Computadora

La precisión se calculó como la proporción de predicciones correctas respecto del total de muestras evaluadas. Debido a que el análisis incluye un modelo generativo, los resultados deben interpretarse como evidencia experimental del desempeño del prototipo y no como garantía de exactitud en cualquier condición de captura.

Se evaluó un conjunto de control compuesto por 40 fotografías de granos de café bajo cuatro condiciones físicas, con 10 muestras por categoría. Al contrastar las predicciones generadas por la inteligencia artificial con una inspección técnica humana, se obtuvieron los resultados mostrados en la Tabla VI.

TABLA VI
Matriz de precisión del diagnóstico visual

Condición del grano evaluada	Muestras de control	Predicciones correctas	Precisión
Secado Óptimo (Color Uniforme)	10	9	90 %
Alta Humedad (Pigmentación Verdosa)	10	8	80 %
Sobreexposición al calor (Tueste oscuro)	10	9	90 %
Presencia de Impurezas (Hojas/Piedras)	10	10	100 %

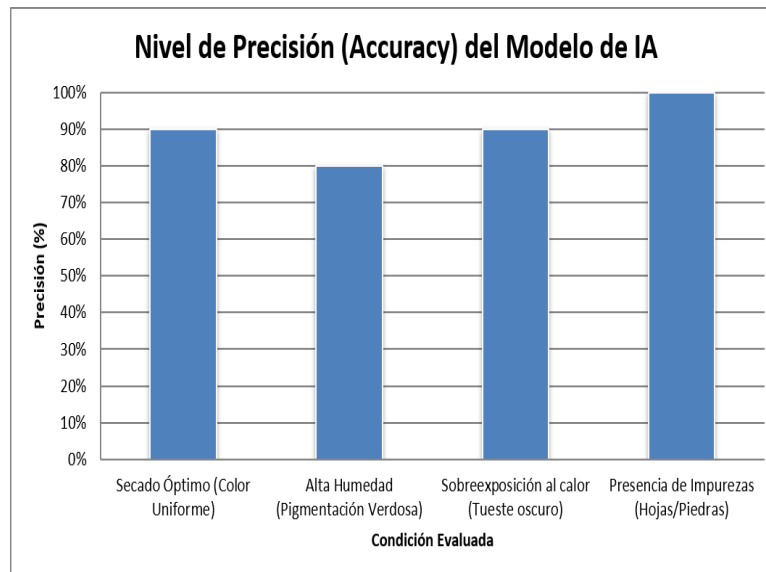


Fig. 9 Nivel de precisión del modelo de IA por condición evaluada.

El sistema alcanzó una precisión global de 90 % (36 aciertos de 40 muestras). La mayor precisión se obtuvo en la identificación de impurezas (100 %), mientras que la condición de alta humedad presentó el menor desempeño (80 %). Esta diferencia puede deberse a que las variaciones de pigmentación verdosa se confunden con sombras o cambios de iluminación. Por ello, una mejora prioritaria consiste en estandarizar la iluminación y la distancia de captura en la bandeja de secado.

IV. ANÁLISIS DE IMPACTO PRELIMINAR

El impacto del sistema se evalúa bajo tres dimensiones fundamentales: técnica, económica y operativa.

A. Impacto Operativo y Calidad del Grano

La automatización del proceso de secado elimina la dependencia de la vigilancia humana constante, la cual es propensa a errores por factores como la fatiga o la inasistencia durante precipitaciones imprevistas. Al integrar un sistema de monitoreo IoT, se asegura que el grano de café mantenga niveles de humedad controlados, reduciendo drásticamente la probabilidad de aparición de moho y fermentación no deseada, factores críticos para la preservación de la inocuidad alimentaria.

B. Impacto en la Toma de Decisiones mediante IA

La introducción de un módulo de diagnóstico basado en IA representa un salto cualitativo en la gestión agrícola. A diferencia de los sistemas tradicionales, el enfoque multimodal propuesto permite que el productor reciba recomendaciones técnicas basadas en evidencia. Esto optimiza la cadena de valor al permitir al agricultor no solo reaccionar ante el clima, sino diagnosticar visualmente la madurez y salud del grano, alineándose con las tendencias actuales de agricultura inteligente ("Smart Agriculture") donde la recolección y procesamiento de grandes volúmenes de datos ambientales facilitan una gestión más eficiente de los recursos.

C. Viabilidad y Escalabilidad

Desde una perspectiva preliminar, el uso de hardware de bajo costo como el ATmega328P democratiza el acceso a tecnología de automatización para pequeños productores que anteriormente no contaban con recursos para implementar sistemas industriales de secado. La proyección hacia una infraestructura IoT más robusta permitiría, en fases futuras, la interconexión de múltiples nodos sensores en campo, generando una red de datos capaz de alimentar modelos predictivos de aprendizaje automático para prever comportamientos climáticos locales.

V. PROYECCIÓN HACIA IoT E INTELIGENCIA ARTIFICIAL

La arquitectura actual del sistema, basada en comunicación Bluetooth de alcance local, constituye la fase inicial de una infraestructura de mayor escala. La proyección del proyecto se enfoca en la transición hacia un ecosistema completo de Internet de las Cosas (IoT) y el fortalecimiento de las capacidades de Inteligencia Artificial (IA).

A. Escalabilidad hacia IoT Global

El sistema presenta una ruta de escalabilidad clara mediante la sustitución del módulo de comunicación local por tecnologías de conectividad de largo alcance, tales como Wi-Fi o redes LPWAN (Low Power Wide Area Network). Esta evolución permitirá que los nodos sensores dispersos en el campo envíen telemetría en tiempo real a una plataforma centralizada en la nube, facilitando la supervisión de múltiples lotes de café simultáneamente desde cualquier ubicación geográfica. La centralización de datos históricos no solo mejorará el control operativo, sino que proporcionará la base necesaria para auditorías de calidad trazables y automatizadas.

B. Evolución hacia Modelos de Aprendizaje Automático

La incorporación actual de IA mediante la API de Gemini para el diagnóstico visual es un primer paso hacia una agricultura predictiva. A futuro, los datos recolectados por la bitácora —temperatura, humedad y patrones de lluvia— podrán conformar un conjunto de datos para entrenar modelos de aprendizaje automático. Estos modelos podrían contribuir a anticipar comportamientos climáticos locales y optimizar los ciclos de secado en función de las condiciones ambientales esperadas, reduciendo la incertidumbre en la toma de decisiones.

VI. CONCLUSIONES

A partir del desarrollo del prototipo y las pruebas experimentales realizadas hasta la fecha, se presentan las siguientes conclusiones:

A. Validación de la Lógica de Control

El uso de una Máquina de Estados Finita (FSM) ha permitido una gestión determinista del sistema, demostrando que es posible automatizar el cierre y apertura de la cobertura con una latencia mecánica de 1.5 a 2.5 segundos, validando la eficacia de la lógica programada sobre el microcontrolador.

B. Diagnóstico Inteligente en Desarrollo

La integración de un módulo de análisis con IA ha permitido transicionar hacia un modelo de diagnóstico preventivo. Los resultados iniciales sugieren que la correlación entre datos ambientales y el análisis visual de la IA proporciona recomendaciones técnicas consistentes, aunque se requiere de un mayor volumen de datos para optimizar la precisión de las sugerencias.

C. Integración IoT y Monitoreo

La implementación de una interfaz móvil mediante Bluetooth ha demostrado ser una solución viable para el monitoreo en tiempo real. La arquitectura permite al productor acceder a una trazabilidad histórica del proceso, factor que resulta determinante para la estandarización del secado.

D. Viabilidad Tecnológica

El uso de hardware basado en Arduino y componentes de bajo costo confirma que la automatización agrícola es accesible para pequeños productores. Estas pruebas iniciales sientan las bases para una futura

fase de optimización, enfocada en mejorar la eficiencia energética del sistema y la estabilidad de la red de comunicación.

LINK VIDEO: “Sistema Autónomo de Protección y Monitoreo de Café con Diagnóstico Inteligente Multimodal”

https://youtu.be/W3_tFDCUEaw?si=1ZBTAgSCvRDWTVzZ

REFERENCIAS

- [1] C. E. Oliveros-Tascón, L. López-Valencia, C. M. Buitrago y E. L. Moreno-Cárdenas, “Determinación del contenido de humedad del café durante el secado en silos,” *Cenicafé*, vol. 61, no. 2, pp. 108-118, 2010.
- [2] G. I. Puerta, “Riesgos para la calidad y la inocuidad del café en el secado,” *Avances Técnicos Cenicafe*, no. 371, pp. 1-8, 2008, doi: 10.38141/10779/0371.
- [3] G. I. Puerta, “La humedad controlada del grano preserva la calidad del café,” *Avances Técnicos Cenicafe*, no. 352, pp. 1-8, 2006.
- [4] A. Sharma, A. Jain, P. Gupta y V. Chowdary, “Machine Learning Applications for Precision Agriculture: A Comprehensive Review,” *IEEE Access*, vol. 9, pp. 4843-4873, 2021, doi: 10.1109/ACCESS.2020.3048415.

APÉNDICE A

FRAGMENTOS DEL CÓDIGO ARDUINO IMPLEMENTADO

Con el fin de complementar la evidencia técnica del prototipo, se presentan a continuación los bloques más representativos del código Arduino implementado. Se muestran únicamente los fragmentos funcionales principales, manteniendo el estilo del informe y evitando incorporar el código como texto plano.

```
1 001: #include <Wire.h>
2 002: #include <LiquidCrystal_I2C.h>
3 003: #include <DHT.h>
4 004: #include <SoftwareSerial.h>
5 005:
6 006: LiquidCrystal_I2C lcd(0x27, 16, 2);
7 007:
8 008: const int pinLluvia = 2;
9 009: const int pinDHT = A0;
10 010:
11 011: #define DHTTYPE DHT11
12 012: DHT dht(pinDHT, DHTTYPE);
13 013:
14 014: const int pinIN1 = 3;
15 015: const int pinIN2 = 4;
16 016: const int pinENA = 5;
17 017:
18 018: // HC-05 TX -> D10. HC-05 RX <- D11 through a voltage divider.
19 019: const int pinBluetoothRX = 10;
20 020: const int pinBluetoothTX = 11;
21 021: SoftwareSerial bluetooth(pinBluetoothRX, pinBluetoothTX);
22 022:
23 023: const int velocidadMotor = 255;
24 024: const unsigned long tiempoGiro = 1000;
25 025: const unsigned long intervaloLectura = 2000;
26 026:
27 027: bool techoAbierto = false;
28 028: bool modoAutomatico = true;
29 029: bool lluviaActual = false;
30 030: unsigned long ultimaLectura = 0;
31 031: String comandoBluetooth = "";
32 032:
33 033: enum EstadoTecho {
34 034:     ABIERTO,
35 035:     CERRADO,
36 036:     CERRADO_LLUVIA,
37 037:     CERRADO_TEMPERATURA,
38 038:     ABIERTO_MANUAL,
39 039:     CERRADO_MANUAL
40 040: };
41 041:
42 042: EstadoTecho estadoTecho = CERRADO;
43 043:
44 044: void setup() {
45 045:     lcd.init();
46 046:     lcd.backlight();
47 047:     dht.begin();
48 048:     bluetooth.begin(9600);
49 049:
50 050:     pinMode(pinLluvia, INPUT);
51 051:     pinMode(pinIN1, OUTPUT);
52 052:     pinMode(pinIN2, OUTPUT);
53 053:     pinMode(pinENA, OUTPUT);
54 054:
55 055:     detenerTecho();
56 056:     mostrarInicio();
```

Fig. 11. Fragmento del código Arduino: configuración general del sistema y función setup().

```

1 059: void loop() {
2 060:     procesarBluetooth();
3 061:
4 062:     if (millis() - ultimaLectura < intervaloLectura) {
5 063:         return;
6 064:     }
7 065:     ultimaLectura = millis();
8 066:
9 067:     const bool hayLluvia = digitalRead(pinLluvia) == LOW;
10 068:     lluviaActual = hayLluvia;
11 069:     const float temperatura = dht.readTemperature();
12 070:     const float humedad = dht.readHumidity();
13 071:
14 072:     if (isnan(temperatura) || isnan(humedad)) {
15 073:         mostrarErrorSensor();
16 074:         bluetooth.println(F("ERROR,DHT11"));
17 075:         return;
18 076:     }
19 077:
20 078:     // Rain remains the highest-priority safety condition, even in manual mode.
21 079:     if (hayLluvia) {
22 080:         cerrarTecho();
23 081:         estadoTecho = CERRADO_LLUVIA;
24 082:     } else if (modoAutomatico) {
25 083:         evaluarModoAutomatico(hayLluvia, temperatura);
26 084:     }
27 085:
28 086:     mostrarEstado(hayLluvia, temperatura, humedad);
29 087:     enviarDatos(temperatura, humedad, hayLluvia);
30 088: }
31 089:
32 090: void evaluarModoAutomatico(bool hayLluvia, float temperatura) {
33 091:     if (temperatura <= 37.0) {
34 092:         abrirTecho();
35 093:         estadoTecho = ABIERTO;
36 094:     } else if (temperatura >= 39.0) {
37 095:         cerrarTecho();
38 096:         estadoTecho = CERRADO_TEMPERATURA;
39 097:     } else {
40 098:         estadoTecho = techoAbierto ? ABIERTO : CERRADO;
41 099:     }
42 100: }

```

Fig. 12. Fragmento del código Arduino: ciclo principal loop() y lógica de control automático.

```

1 102: void procesarBluetooth() {
2 103:   while (bluetooth.available()) {
3 104:     const char caracter = bluetooth.read();
4 105:
5 106:     if (caracter == '\n' || caracter == '\r') {
6 107:       if (comandoBluetooth.length() > 0) {
7 108:         ejecutarComando(comandoBluetooth);
8 109:         comandoBluetooth = "";
9 110:       }
10 111:     } else if (comandoBluetooth.length() < 16) {
11 112:       comandoBluetooth += caracter;
12 113:     }
13 114:   }
14 115: }
15 116:
16 117: void ejecutarComando(String comando) {
17 118:   comando.trim();
18 119:   comando.toUpperCase();
19 120:
20 121:   if (comando == "A") {
21 122:     if (lluviaActual) {
22 123:       bluetooth.println(F("ACK,BLOCKED_RAIN"));
23 124:       return;
24 125:     }
25 126:     modoAutomatico = false;
26 127:     abrirTecho();
27 128:     estadoTecho = ABIERTO_MANUAL;
28 129:     bluetooth.println(F("ACK,OPEN"));
29 130:   } else if (comando == "C") {
30 131:     modoAutomatico = false;
31 132:     cerrarTecho();
32 133:     estadoTecho = CERRADO_MANUAL;
33 134:     bluetooth.println(F("ACK,CLOSE"));
34 135:   } else if (comando == "M") {
35 136:     modoAutomatico = true;
36 137:     bluetooth.println(F("ACK,AUTO"));
37 138:   }
38 139: }
39 140:
40 141: void enviarDatos(float temperatura, float humedad, bool hayLluvia) {
41 142:   bluetooth.print(F("DATA,"));
42 143:   bluetooth.print(temperatura, 1);
43 144:   bluetooth.print(',');
44 145:   bluetooth.print(humedad, 1);
45 146:   bluetooth.print(',');
46 147:   bluetooth.print(hayLluvia ? 1 : 0);
47 148:   bluetooth.print(',');
48 149:   bluetooth.print(nombreEstado(estadoTecho));
49 150:   bluetooth.print(',');
50 151:   bluetooth.println(modoAutomatico ? F("AUTO") : F("MANUAL"));
51 152: }
52 153:
53 154: const char *nombreEstado(EstadoTecho estado) {
54 155:   switch (estado) {
55 156:     case ABIERTO:
56 157:       return "OPEN";
57 158:     case CERRADO_LLUVIA:
58 159:       return "CLOSED_RAIN";
59 160:     case CERRADO_TEMPERATURA:
60 161:       return "CLOSED_TEMP";
61 162:     case ABIERTO_MANUAL:
62 163:       return "OPEN_MANUAL";
63 164:     case CERRADO_MANUAL:
64 165:       return "CLOSED_MANUAL";
65 166:     default:
66 167:       return "CLOSED";
67 168:   }
68 169: }

```

Fig. 13. Fragmento del código Arduino: procesamiento de comandos Bluetooth y envío de datos.

```

1 171: void abrirTecho() {
2 172:     if (techoAbierto) {
3 173:         return;
4 174:     }
5 175:
6 176:     analogWrite(pinENA, velocidadMotor);
7 177:     digitalWrite(pinIN1, HIGH);
8 178:     digitalWrite(pinIN2, LOW);
9 179:     delay(tiempoGiro);
10 180:     detenerTecho();
11 181:     techoAbierto = true;
12 182: }
13 183:
14 184: void cerrarTecho() {
15 185:     if (!techoAbierto) {
16 186:         return;
17 187:     }
18 188:
19 189:     analogWrite(pinENA, velocidadMotor);
20 190:     digitalWrite(pinIN1, LOW);
21 191:     digitalWrite(pinIN2, HIGH);
22 192:     delay(tiempoGiro);
23 193:     detenerTecho();
24 194:     techoAbierto = false;
25 195: }
26 196:
27 197: void detenerTecho() {
28 198:     analogWrite(pinENA, 0);
29 199:     digitalWrite(pinIN1, LOW);
30 200:     digitalWrite(pinIN2, LOW);
31 201: }
32 202:
33 203: void mostrarInicio() {
34 204:     lcd.clear();
35 205:     lcd.setCursor(0, 0);
36 206:     lcd.print(F("SISTEMA CAFE"));
37 207:     lcd.setCursor(0, 1);
38 208:     lcd.print(F("MODO AUTOMATICO"));
39 209: }
40 210:
41 211: void mostrarErrorSensor() {
42 212:     lcd.clear();
43 213:     lcd.setCursor(0, 0);
44 214:     lcd.print(F("ERROR LECTURA"));
45 215:     lcd.setCursor(0, 1);
46 216:     lcd.print(F("SENSOR DHT11"));
47 217: }
48 218:
49 219: void mostrarEstado(bool hayLluvia, float temperatura, float humedad) {
50 220:     lcd.clear();
51 221:
52 222:     if (hayLluvia) {
53 223:         lcd.setCursor(0, 0);
54 224:         lcd.print(F("LLUVIA DETECT."));
55 225:         lcd.setCursor(0, 1);
56 226:         lcd.print(F("TECHO CERRADO"));
57 227:         return;
58 228:     }
59 229:
60 230:     lcd.setCursor(0, 0);
61 231:     lcd.print(F("T:"));
62 232:     lcd.print(temperatura, 1);
63 233:     lcd.print(F(" H:"));
64 234:     lcd.print(humedad, 0);
65 235:     lcd.print('%');
66 236:
67 237:     lcd.setCursor(0, 1);
68 238:     lcd.print(modoAutomatico ? F("AUTO ") : F("MANUAL "));
69 239:     lcd.print(techoAbierto ? F("ABIERTO") : F("CERRADO"));
70 240: }

```

Fig. 14. Fragmento del código Arduino: accionamiento del techo y visualización en pantalla LCD.